

Linux Aygıt Sürücüsü

Geliştirme Rehberi

Çekirdek Modüllerinden Aygıt Modeline Uygulamalı Başvuru

Bu belge, Linux çekirdek modülü ve aygıt sürücüsü geliştirmeyi **çalışan kod örnekleriyle** baştan sona anlatır. İlk “merhaba dünya” modülünden karakter aygıtlarına, kesme işleyicilerinden aygıt modeline ve platform sürücülerine kadar sürücü geliştirmenin temellerini kapsar.

Kısım 1 — Çekirdek Modülü Temelleri: kullanıcı alanı vs çekirdek • ilk modül • Makefile & derleme • modül parametreleri • printk & günlükleme.

Kısım 2 — Karakter Aygıtları: aygıt dosyaları & major/minor • cdev & file_operations • open/read/write • copy_to/from_user • ioctl • udev & otomatik aygıt düğümü.

Kısım 3 — Çekirdek Altyapısı: bellek ayırma • eşzamanlılık (spinlock, mutex, atomik) • kesmeler & alt yarı • zaman & zamanlayıcılar.

Kısım 4 — Aygıt Modeli: sysfs & kobject • platform sürücüler • device tree • kaynak yönetimi.

Kısım 5 — İleri Konular: bekleme kuyrukları & bloklamalı G/Ç • poll • proc arayüzleri • hata ayıklama.

Hazırlanma Tarihi: 12 Temmuz 2026

Linux Çekirdek 5.x/6.x • C • Kodlar İngilizce, açıklamalar Türkçe

Contents

1 Giriş: Sürücüler ve Çekirdek Alanı	2
1.1 Kullanıcı Alanı vs Çekirdek Alanı	2
1.2 Statik Derleme vs Yüklenebilir Modül	2
1.3 Geliştirme Ortamının Hazırlanması	3
2 İlk Çekirdek Modülü	3
2.1 Modülün Kaynak Kodu	3
2.2 Makefile ve Derleme	4
2.3 Modülü Yükleme ve Test Etme	4
3 Günlükleme ve Modül Parametreleri	4
3.1 printk ve Günlük Seviyeleri	4
3.2 Modül Parametreleri	5
4 Aygıt Dosyaları ve Aygıt Numaraları	7
4.1 Karakter vs Blok Aygıtları	7
4.2 Major ve Minor Numaraları	7
5 Karakter Sürücüsü: file_operations	8
5.1 file_operations Yapısı	8
5.2 cdev ile Aygıtı Kaydetme	8
6 Okuma, Yazma ve Kullanıcı Verisi	9
6.1 copy_to_user ve copy_from_user	9
6.2 open ve release	10
6.3 Sürücüyü Test Etme	11
7 ioctl: Aygıt Kontrol Komutları	11
7.1 ioctl Komutlarını Tanımlama	11
7.2 Kullanıcı Alanından ioctl Çağırma	12
8 Otomatik Aygıt Düğümü: udev ve class	12
9 Çekirdekte Bellek Yönetimi	14
9.1 kmalloc, kzalloc ve vmalloc	14
9.2 Kaynak Yönetimli (devm) Ayırma	15
10 Eşzamanlılık ve Kilitleme	15
10.1 Yarış Koşulu Neden Oluşur?	15
10.2 Mutex	15
10.3 Spinlock	16
10.4 Atomik İşlemler	16
11 Kesmeler (Interrupts)	17
11.1 Kesme İşleyicisi Kaydetme	17
11.2 Üst Yarı ve Alt Yarı (Top/Bottom Half)	17
12 Çekirdekte Zaman	18
12.1 jiffies ve Gecikmeler	18
12.2 Çekirdek Zamanlayıcıları	19

13 sysfs ile Kullanıcı Arayüzü	20
13.1 sysfs Niteliği Oluşturma	20
14 Platform Sürücüler	21
14.1 Platform Sürücüsünün İskeleti	21
15 Device Tree ve Kaynaklar	22
15.1 Device Tree Düzümü	22
15.2 probe İçinde Kaynak Alma	22
15.3 Donanım Register'larına Erişim (MMIO)	23
16 Bekleme Kuyrukları ve Bloklamalı G/Ç	24
16.1 Bekleme Kuyruğu ile Uyutma ve Uyandırma	24
16.2 poll/select Desteği	25
17 Hata Ayıklama Teknikleri	25
17.1 Kernel Oops ve Panic Okuma	25
17.2 debugfs ile İç Durum Sunma	26
18 Kapanış: Sürücü Geliştirme Yol Haritası	26

KISIM 1

Çekirdek Modülü Temelleri

1. Giriş: Sürücüler ve Çekirdek Alanı

Aygıt sürücüsü (device driver), işletim sisteminin donanım ile konuşmasını sağlayan yazılım katmanıdır. Uygulamalar donanımı doğrudan bilmez; sürücüler soyutlama sağlar — bir uygulama `read()` çağırıldığında, ister bir diskten ister bir ağ kartından okusun aynı arayüzü kullanır. Linux'ta sürücülerin çoğu *çekirdek modülü* olarak yazılır.

1.1 Kullanıcı Alanı vs Çekirdek Alanı

Linux belleği iki ayrı bölgeye ayırır. Sürücü yazarken sürekli bu ayrımın farkında olmalısın, çünkü çekirdekte yapılan bir hata tüm sistemi çökertebilir.

	Kullanıcı Alanı	Çekirdek Alanı
Çalışan kod	Uygulamalar	Çekirdek, modüller, sürücüler
Ayrıcalık	Kısıtlı (ring 3)	Tam yetki (ring 0)
Hata sonucu	Süreç çöker	Sistem çöker (kernel panic)
Bellek erişimi	Kendi sanal alanı	Tüm fiziksel bellek
Kütüphaneler	libc, tüm kullanıcı kütüph.	Yalnızca çekirdek API'si

Dikkat

Çekirdek alanında **standart C kütüphanesi yoktur**: `printf`, `malloc`, `stdio.h` kullanamazsın. Bunların yerine çekirdek API'si gelir: `printk`, `kmalloc`, `<linux/*.h>` başlıkları. Ayrıca kayan nokta (float) işlemleri de kural olarak kullanılmaz. Çekirdek kısıtlı ve kendine özgü bir ortamdır.

1.2 Statik Derleme vs Yüklenebilir Modül

Bir sürücü çekirdeğe iki şekilde dahil edilebilir:

Yöntem	Açıklama
Statik (built-in)	Çekirdeğin içine derlenir; her zaman yüklüdür. Yeniden derleme gerekir.
Modül (.ko)	Çalışırken <code>insmod</code> ile yüklenir, <code>rmmmod</code> ile kaldırılır. Geliştirme için idealdir.

Geliştirme sırasında neredeyse her zaman *yüklenebilir çekirdek modülü* (Loadable Kernel Module — LKM) kullanırız; çünkü çekirdeği yeniden başlatmadan modülü yükleyip kaldırabiliriz.

1.3 Geliştirme Ortamının Hazırlanması

Modül derlemek için çalışan çekirdeğinizle eşleşen başlık dosyaları gerekir.

```

1 # Derleme araçları ve çekirdek başlıkları (Debian/Ubuntu)
2 $ sudo apt install build-essential linux-headers-$(uname -r)
3
4 # Fedora/RHEL
5 $ sudo dnf install kernel-devel kernel-headers gcc make
6
7 # Çalışan çekirdek sürümünü gör
8 $ uname -r                # ör. 6.8.0-40-generic
9
10 # Başlıkların yeri (Makefile bunu kullanır)
11 $ ls /lib/modules/$(uname -r)/build

```

Bilgi

Sürücü geliştirmeyi **sanal makinede** (QEMU/VirtualBox) yapmak şiddetle önerilir: çekirdek kodundaki bir hata sistemi çökertebilir veya veri kaybına yol açabilir. Sanal makinede çökme olsa bile ana sisteminiz güvende kalır ve anlık görüntülerden (snapshot) hızlıca geri dönebilirsiniz.

2. İlk Çekirdek Modülü

Geleneksel “merhaba dünya” ile başlayalım. Bir çekirdek modülünün iki temel fonksiyonu vardır: yüklendiğinde çalışan bir *giriş* (init) fonksiyonu ve kaldırıldığında çalışan bir *çıkış* (exit) fonksiyonu.

2.1 Modülün Kaynak Kodu

```

1 #include <linux/module.h>      /* tüm modüller için gerekli */
2 #include <linux/kernel.h>      /* printk, KERN_INFO */
3 #include <linux/init.h>        /* __init, __exit makroları */
4
5 /* Modül yüklendiğinde çağrılır. __init: bu kod yalnızca bir kez
6    çalışır, sonra belleği serbest bırakılabilir. */
7 static int __init hello_init(void)
8 {
9     printk(KERN_INFO "Merhaba: modül yüklendi\n");
10    return 0;                  /* 0 = başarı; negatif = hata (yükleme iptal) */
11 }
12
13 /* Modül kaldırıldığında çağrılır. */
14 static void __exit hello_exit(void)
15 {
16     printk(KERN_INFO "Güle güle: modül kaldırıldı\n");
17 }
18
19 module_init(hello_init);      /* giriş noktasını kaydet */
20 module_exit(hello_exit);      /* çıkış noktasını kaydet */
21
22 MODULE_LICENSE("GPL");        /* lisans; GPL olmayan modüller çekirdeği "kirletir" */
23 MODULE_AUTHOR("Ad Soyad");
24 MODULE_DESCRIPTION("Basit bir merhaba dünya modülü");

```

```
25 MODULE_VERSION("1.0");
```

Dikkat

MODULE_LICENSE zorunludur. GPL uyumlu bir lisans belirtmezsen çekirdek “tainted” (kirlenmiş) olarak işaretlenir ve birçok GPL-only çekirdek sembolüne (fonksiyona) erişemezsin. Ayrıca giriş/çıkış fonksiyonları static olmalıdır — global ad alanını kirletmemek için.

2.2 Makefile ve Derleme

Çekirdek modülleri, çekirdeğin kendi *kbuild* sistemiyle derlenir. Makefile şaşırtıcı derecede kısadır:

```
1 # Makefile
2 obj-m += hello.o           # 'hello.c' -> 'hello.ko' üret
3
4 # Çalışan çekirdeğin derleme dizinine yönlendir
5 KDIR := /lib/modules/$(shell uname -r)/build
6
7 all:
8     $(MAKE) -C $(KDIR) M=$(PWD) modules
9
10 clean:
11     $(MAKE) -C $(KDIR) M=$(PWD) clean
```

Kbuild nasıl çalışır?

obj-m += hello.o satırı kbuild’e “hello.c’yi yüklenebilir bir modül olarak derle” der. -C \$(KDIR) çekirdeğin Makefile’ına geçer, M=\$(PWD) ise “modül kaynağı burada” anlamına gelir. Sonuç .ko (kernel object) uzantılı bir dosyadır.

2.3 Modülü Yükleme ve Test Etme

```
1 $ make                # hello.ko üret
2 $ sudo insmod hello.ko # modülü yükle -> hello_init çalışır
3 $ lsmod | grep hello   # yüklü modülleri listele
4 $ sudo rmmod hello     # modülü kaldır -> hello_exit çalışır
5
6 $ dmesg | tail         # çekirdek günlük mesajlarını gör
7 # [ ... ] Merhaba: modül yüklendi
8 # [ ... ] Güle güle: modül kaldırıldı
9
10 $ modinfo hello.ko     # modül meta bilgisini gör (lisans, yazar...)
```

3. Günlükleme ve Modül Parametreleri

3.1 printk ve Günlük Seviyeleri

Çekirdekte printf yerine printk kullanılır. printk, mesajlara bir *öncelik seviyesi* ekler; bu seviye mesajın nereye ve ne zaman gösterileceğini etkiler.

Makro	Seviye	Kullanım
KERN_EMERG	0	Sistem kullanılamaz.
KERN_ERR	3	Hata durumu.
KERN_WARNING	4	Uyarı.
KERN_INFO	6	Bilgilendirme.
KERN_DEBUG	7	Hata ayıklama.

```

1 /* Modern çekirdeklerde tercih edilen yardımcılar (pr_* makroları) */
2 pr_info("aygıt %d başlatıldı\n", id);          /* = printk(KERN_INFO ...) */
3 pr_err("bellek ayrılamadı\n");                 /* = printk(KERN_ERR ...) */
4 pr_warn("tampon neredeyse dolu\n");
5 pr_debug("hata ayıklama: sayaç=%d\n", count); /* yalnızca DEBUG etkinse */
6
7 /* Bir sürücü içinde daha iyisi: dev_* (aygıtı da yazdırır) */
8 dev_info(dev, "başlatma tamam\n");
9 dev_err(dev, "kayıt başarısız: %d\n", ret);

```

İpucu

dmesg çekirdek halka tamponunu (ring buffer) gösterir; sürücü geliştirmede en çok kullanacağın araçtır. Canlı izlemek için dmesg -w (watch) kullan. printk biçim belirteçleri printf'e benzer ama çekirdeğe özgü olanlar da vardır: işaretçi için %p, güvenli hash'li işaretçi; fiziksel adres için %pa; resource için %pr.

3.2 Modül Parametreleri

Modüllere, yükleme anında dışarıdan değer geçirebiliriz — tıpkı bir programa komut satırı argümanı geçmek gibi. Bu, aynı modülü farklı ayarlarla kullanmayı sağlar.

```

1 #include <linux/moduleparam.h>
2
3 static int buffer_size = 1024;          /* varsayılan değer */
4 static char *device_name = "mydev";
5 static int debug_enabled = 0;
6
7 /* Parametreyi kaydet: (değişken, tip, sysfs izinleri) */
8 module_param(buffer_size, int, 0644);
9 MODULE_PARM_DESC(buffer_size, "Tampon boyutu (bayt)");
10
11 module_param(device_name, charp, 0644); /* charp = char pointer */
12 MODULE_PARM_DESC(device_name, "Aygıt adı");
13
14 module_param(debug_enabled, bool, 0644);
15 MODULE_PARM_DESC(debug_enabled, "Hata ayıklama günlüğünü aç (0/1)");
16
17 static int __init my_init(void)
18 {
19     pr_info("tampon=%d ad=%s hata_ayıkla=%d\n",
20           buffer_size, device_name, debug_enabled);
21     return 0;
22 }

```

```
1 # Parametreleri yükleme anında geç
2 $ sudo insmod mymod.ko buffer_size=4096 device_name="sensor" debug_enabled=1
3
4 # Yüklü bir modülün parametrelerini sysfs'ten oku (0644 izinleri sayesinde)
5 $ cat /sys/module/mymod/parameters/buffer_size
6 4096
```

Bilgi

module_param'ın üçüncü argümanı, parametrenin /sys/module/.../parameters/ altındaki dosya izinleridir. 0644 ile parametre çalışırken okunabilir (ve root tarafından yazılabilir); 0 verirken sysfs'te hiç görünmez. Böylece bir sürücünün davranışını yeniden yüklemekten ayarlayabilirsin.

KISIM 2

Karakter Aygıtları

4. Aygıt Dosyaları ve Aygıt Numaraları

Linux'ta “her şey bir dosyadır” ilkesi donanım için de geçerlidir. Aygıtlar, /dev altındaki özel dosyalar aracılığıyla kullanıcı alanına sunulur. Bir uygulama bu dosyaya `open`, `read`, `write` yaptığında, çekirdek çağrısı ilgili sürücüye yönlendirir.

4.1 Karakter vs Blok Aygıtları

Tür	Erişim	Örnek
Karakter (char)	Bayt akışı, sırayla	Seri port, klavye, /dev/null
Blok (block)	Sabit boyutlu bloklar, rastgele	Disk, SSD, USB bellek
Ağ (network)	Paket, soket arayüzü	Ethernet, Wi-Fi

Bu rehber *karakter aygıtlarına* odaklanır; çünkü en yaygın ve öğrenmesi en kolay sürücü türüdür. Sensörler, GPIO, seri portlar genelde karakter aygıtı olarak sunulur.

4.2 Major ve Minor Numaraları

Her aygıt dosyası iki sayıyla tanımlanır: *major* numara hangi sürücünün sorumlu olduğunu, *minor* numara ise o sürücünün hangi somut aygıtı (birden çok aynı tip aygıt olabilir) belirtir.

```

1 $ ls -l /dev/null /dev/ttyS0
2 crw-rw-rw- 1 root root 1, 3 ... /dev/null    # major=1, minor=3
3 crw-rw---- 1 root dialout 4, 64 ... /dev/ttyS0 # major=4, minor=64
4 # ^ 'c' = karakter aygıtı (block için 'b')
```

```

1 #include <linux/fs.h>
2
3 dev_t dev;                                /* aygıt numarası (major+minor) */
4
5 /* Çekirdekten dinamik olarak bir major numara iste (önerilen) */
6 ret = alloc_chrdev_region(&dev, 0, 1, "mydev");
7 /* &dev: sonuç, 0: ilk minor, 1: aygıt sayısı, "mydev": ad */
8
9 int major = MAJOR(dev);                   /* numaradan major çıkar */
10 int minor = MINOR(dev);                  /* numaradan minor çıkar */
```

```

11 pr_info("major=%d minor=%d\n", major, minor);
12
13 /* Kullanım sonrası serbest bırak (exit fonksiyonunda) */
14 unregister_chrdev_region(dev, 1);

```

İpucu

Eskiden sabit major numaralar (`register_chrdev`) kullanılırdı ama çakışma riski vardır. Modern sürücüler `alloc_chrdev_region` ile çekirdekten *dinamik* numara ister — böylece sistemdeki başka bir sürücüyle çakışmaz.

5. Karakter Sürücüsü: file_operations

Bir karakter sürücüsünün kalbi `struct file_operations`'tır: kullanıcı alanı sistem çağrılarını (`open`, `read`, `write`...) senin sürücü fonksiyonlarına bağlayan bir işaretçi tablosudur. Çekirdek, uygulama aygıt dosyasına bir işlem yaptığında ilgili fonksiyonu çağırır.

5.1 file_operations Yapısı

```

1  #include <linux/fs.h>
2
3  static int      my_open(struct inode *inode, struct file *file);
4  static int      my_release(struct inode *inode, struct file *file);
5  static ssize_t  my_read(struct file *file, char __user *buf,
6                        size_t count, loff_t *offset);
7  static ssize_t  my_write(struct file *file, const char __user *buf,
8                        size_t count, loff_t *offset);
9
10 static const struct file_operations my_fops = {
11     .owner      = THIS_MODULE,      /* modül sayacı yönetimi için */
12     .open       = my_open,
13     .release    = my_release,      /* close() sistem çağrısına karşılık gelir */
14     .read       = my_read,
15     .write      = my_write,
16     .llseek     = default_llseek,
17 };

```

__user ne anlama gelir?

`char __user *buf` imzasındaki `__user`, bu işaretçinin *kullanıcı alanına* ait olduğunu belirtir. Çekirdek bu işaretçiyi doğrudan dereference **edemez** — geçerliliği garanti değildir ve güvenlik açığı olur. Bunun yerine `copy_to_user` / `copy_from_user` kullanılır. `sparse` statik analiz aracı bu işaretlemeyi denetler.

5.2 cdev ile Aygıtı Kaydetme

Aygıt numarasını aldıktan sonra, `struct cdev` ile `file_operations` tablosunu çekirdeğe kaydederiz. Böylece o major/minor'a yapılan çağrılar bizim fonksiyonlarımıza gelir.

```

1  #include <linux/cdev.h>
2
3  static struct cdev my_cdev;          /* karakter aygıtı yapısı */
4
5  static int __init my_init(void)

```

```

6 {
7     int ret;
8
9     /* 1) Aygıt numarası ayır */
10    ret = alloc_chrdev_region(&dev, 0, 1, "mydev");
11    if (ret < 0) {
12        pr_err("aygıt numarası ayrılamadı\n");
13        return ret;
14    }
15
16    /* 2) cdev'i file_operations ile ilklendir ve kaydet */
17    cdev_init(&my_cdev, &my_fops);
18    my_cdev.owner = THIS_MODULE;
19    ret = cdev_add(&my_cdev, dev, 1);      /* çekirdeğe ekle */
20    if (ret < 0) {
21        unregister_chrdev_region(dev, 1);  /* hata: geri al */
22        return ret;
23    }
24
25    pr_info("sürücü hazır (major=%d)\n", MAJOR(dev));
26    return 0;
27 }
28
29 static void __exit my_exit(void)
30 {
31     cdev_del(&my_cdev);                  /* kaydı kaldır */
32     unregister_chrdev_region(dev, 1);    /* numarayı serbest bırak */
33 }

```

Dikkat

Çekirdek programlamada **hata yönetimi ve temizlik** kritiktir. Bir adım başarısız olursa, o ana kadar ayrılan tüm kaynakları *ters sırayla* serbest bırakmalısın (yukarıdaki örnekte `alloc_chrdev_region` başarılı ama `cdev_add` başarısızsa, numarayı geri veriyoruz). Sızdırılan çekirdek kaynağı, yeniden başlatmaya kadar geri gelmez.

6. Okuma, Yazma ve Kullanıcı Verisi

Sürücünün asıl işi `read` ve `write` fonksiyonlarındadır. Buradaki en önemli kural: çekirdek ile kullanıcı alanı arasında veri, asla doğrudan işaretçiyle değil, özel fonksiyonlarla taşınır.

6.1 `copy_to_user` ve `copy_from_user`

```

1  #include <linux/uaccess.h>      /* copy_to_user, copy_from_user */
2  #include <linux/slab.h>         /* kmalloc, kfree */
3
4  #define BUF_SIZE 1024
5  static char *device_buffer;     /* çekirdek içi tampon */
6  static size_t buffer_len;       /* içindeki geçerli bayt sayısı */
7
8  /* Kullanıcı aygıttan OKUYOR: veri çekirdek -> kullanıcı yönünde akar */
9  static ssize_t my_read(struct file *file, char __user *buf,
10                        size_t count, loff_t *offset)
11 {
12     size_t remaining;

```

```

13     if (*offset >= buffer_len)                /* dosya sonu */
14         return 0;
15
16     /* İstenen kadar ama en fazla kalan veri kadar oku */
17     if (count > buffer_len - *offset)
18         count = buffer_len - *offset;
19
20     /* Çekirdek tamponundan kullanıcı tamponuna kopyala.
21        Dönüş: KOPYALANAMAYAN bayt sayısı (0 = tam başarı). */
22     remaining = copy_to_user(buf, device_buffer + *offset, count);
23     if (remaining)
24         return -EFAULT;                        /* geçersiz kullanıcı adresi */
25
26     *offset += count;                          /* konumu ilerlet */
27     pr_debug("%zu bayt okundu\n", count);
28     return count;                             /* okunan bayt sayısını döndür */
29 }
30
31 /* Kullanıcı aygıtı YAZIYOR: veri kullanıcı -> çekirdek yönünde akar */
32 static ssize_t my_write(struct file *file, const char __user *buf,
33                        size_t count, loff_t *offset)
34 {
35     if (count > BUF_SIZE)                     /* taşmayı önle */
36         count = BUF_SIZE;
37
38     if (copy_from_user(device_buffer, buf, count))
39         return -EFAULT;
40
41     buffer_len = count;
42     pr_debug("%zu bayt yazıldı\n", count);
43     return count;
44 }
45

```

Dikkat

copy_to_user/copy_from_user başarısızlıkta kopyalanamayan bayt sayısını döndürür, hata kodu değil. Sıfır dönmesi tam başarı demektir. Ayrıca bu fonksiyonlar uyuyabilir (sayfa hatası tetikleyebilir), bu yüzden atomik bağlamda (spinlock tutarken, kesme işleyicide) çağrılmazlar. Doğrudan *buf = x yapmak ise ciddi bir güvenlik açığı ve çökme sebebidir.

6.2 open ve release

```

1 static int my_open(struct inode *inode, struct file *file)
2 {
3     /* Tampon henüz yoksa ayır (basit örnek; genelde init'te yapılır) */
4     if (!device_buffer) {
5         device_buffer = kmalloc(BUF_SIZE, GFP_KERNEL);
6         if (!device_buffer)
7             return -ENOMEM;                  /* bellek yok */
8     }
9     pr_info("aygıt açıldı\n");
10    return 0;
11 }
12
13 static int my_release(struct inode *inode, struct file *file)
14 {

```

```

15     pr_info("aygıt kapatıldı\n");
16     return 0;
17 }

```

6.3 Sürücüyü Test Etme

```

1 # Aygıt düğümünü elle oluştur (major numarayı dmesg'den al)
2 $ sudo mknod /dev/mydev c 240 0      # c=karakter, 240=major, 0=minor
3 $ sudo chmod 666 /dev/mydev
4
5 # Yaz ve oku
6 $ echo "merhaba sürücü" > /dev/mydev
7 $ cat /dev/mydev
8 merhaba sürücü
9
10 # Düşük seviye test
11 $ dd if=/dev/mydev bs=1 count=10

```

7. ioctl: Aygıt Kontrol Komutları

read/write bayt akışı içindir, ama aygıtlara çoğu zaman *komut* göndermemiz gerekir: “baud hızını ayarla”, “sıfırla”, “durumu sorgula”. Bunun için `ioctl` (input/output control) kullanılır.

7.1 ioctl Komutlarını Tanımlama

Komut numaraları rastgele seçilmez; çakışmayı önleyen makrolarla üretilir. Bu makrolar komuta bir yön (okuma/yazma) ve veri boyutu da kodlar.

```

1 #include <linux/ioctl.h>
2
3 #define MYDEV_MAGIC 'k'                /* sürücüye özel "sihirli" harf */
4
5 /* _IO: veri yok, _IOR: oku, _IOW: yaz, _IOWR: her ikisi */
6 #define MYDEV_RESET      _IO(MYDEV_MAGIC, 0)
7 #define MYDEV_GET_SIZE   _IOR(MYDEV_MAGIC, 1, int)
8 #define MYDEV_SET_SIZE   _IOW(MYDEV_MAGIC, 2, int)
9
10 static long my_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
11 {
12     int value;
13
14     switch (cmd) {
15     case MYDEV_RESET:
16         buffer_len = 0;                /* tamponu sıfırla */
17         memset(device_buffer, 0, BUF_SIZE);
18         pr_info("aygıt sıfırlandı\n");
19         break;
20
21     case MYDEV_GET_SIZE:                /* çekirdek -> kullanıcı */
22         value = buffer_len;
23         if (copy_to_user((int __user *)arg, &value, sizeof(value)))
24             return -EFAULT;
25         break;
26
27     case MYDEV_SET_SIZE:                /* kullanıcı -> çekirdek */

```

```

28     if (copy_from_user(&value, (int __user *)arg, sizeof(value)))
29         return -EFAULT;
30     if (value < 0 || value > BUF_SIZE)
31         return -EINVAL;           /* geçersiz argüman */
32     buffer_len = value;
33     break;
34
35     default:
36         return -ENOTTY;           /* bilinmeyen komut */
37 }
38 return 0;
39 }
40
41 /* file_operations'a ekle: */
42 /* .unlocked_ioctl = my_ioctl, */

```

7.2 Kullanıcı Alanından ioctl Çağırma

```

1  /* Kullanıcı alanı programı (aynı komut tanımlarını paylaşır) */
2  #include <sys/ioctl.h>
3  #include <fcntl.h>
4
5  int fd = open("/dev/mydev", O_RDWR);
6
7  ioctl(fd, MYDEV_RESET);          /* argümansız komut */
8
9  int size = 512;
10 ioctl(fd, MYDEV_SET_SIZE, &size); /* değer gönder */
11
12 int current;
13 ioctl(fd, MYDEV_GET_SIZE, &current); /* değer al */
14 printf("mevcut boyut: %d\n", current);
15
16 close(fd);

```

Bilgi

ioctl güçlüdür ama tip güvenliği zayıftır (her şey unsigned long arg'dan geçer). Modern çekirdek geliştirmede, basit durum sorguları için ioctl yerine sysfs nitelikleri (Kısım 4) tercih edilir — çünkü kabuktan cat/echo ile erişilebilir ve kendini belgeler. Yine de karmaşık, ikili protokoller için ioctl vazgeçilmezdir.

8. Otomatik Aygıt Düğümü: udev ve class

Yukarıda aygıt düğümünü elle mknod ile oluşturduk — bu zahmetli ve hataya açıktır. Gerçek sürücüler, çekirdeğin *aygıt modeli* üzerinden bir class ve device kaydeder; *udev* arka planı bunu görüp /dev altında düğümü **otomatik** oluşturur.

```

1  #include <linux/device.h>
2
3  static struct class *my_class;
4  static struct device *my_device;
5
6  static int __init my_init(void)

```

```

7 {
8     /* ... alloc_chrdev_region + cdev_add (yukarıdaki gibi) ... */
9
10    /* 1) Bir aygıt sınıfı oluştur -> /sys/class/myclass/ */
11    my_class = class_create("myclass");      /* çekirdek 6.4+ imzası */
12    if (IS_ERR(my_class)) {
13        cdev_del(&my_cdev);
14        unregister_chrdev_region(dev, 1);
15        return PTR_ERR(my_class);
16    }
17
18    /* 2) Aygıtı oluştur -> udev /dev/mydev düğümünü otomatik yapar */
19    my_device = device_create(my_class, NULL, dev, NULL, "mydev");
20    if (IS_ERR(my_device)) {
21        class_destroy(my_class);
22        cdev_del(&my_cdev);
23        unregister_chrdev_region(dev, 1);
24        return PTR_ERR(my_device);
25    }
26
27    pr_info("/dev/mydev otomatik oluşturuldu\n");
28    return 0;
29 }
30
31 static void __exit my_exit(void)
32 {
33     device_destroy(my_class, dev);          /* ters sırayla temizle */
34     class_destroy(my_class);
35     cdev_del(&my_cdev);
36     unregister_chrdev_region(dev, 1);
37 }

```

IS_ERR ve PTR_ERR

Birçok çekirdek fonksiyonu, hata durumunda NULL yerine bir *hata işaretçisi* döndürür (adres uzayının en üstündeki özel değerler). IS_ERR(ptr) bunun bir hata olup olmadığını kontrol eder, PTR_ERR(ptr) ise onu bir hata koduna (-ENOMEM gibi) çevirir. Bu, çekirdekte çok yaygın bir kalıptır; işaretçi dönen fonksiyonlarda mutlaka kontrol et.

Bilgi

Bu noktada tam işlevsel bir karakter sürücün var: dinamik major numara, cdev kaydı, read/write/ioctl ve udev ile otomatik /dev düğümü. Bu iskelet, gerçek sürücülerin çoğunun temelini oluşturur — geri kalanı, read/write içinde gerçek donanımla konuşmaktan ibarettir.

KISIM 3

Çekirdek Altyapısı

9. Çekirdekte Bellek Yönetimi

Çekirdekte malloc yoktur; bellek ayırmanın kendine özgü fonksiyonları ve kuralları vardır. Yanlış ayırma bayrağı veya sızıntı, tüm sistemi etkileyebilir — kullanıcı alanındaki gibi süreç bitince otomatik temizlik yoktur.

9.1 kmalloc, kzalloc ve vmalloc

Fonksiyon	Özellik
kmalloc	Fiziksel olarak <i>bitişik</i> bellek; küçük, hızlı; DMA'ya uygun.
kzalloc	kmalloc + sıfırlama (önerilir).
vmalloc	Sanal olarak bitişik (fiziksel dağınık); büyük ayırmalar için.
kfree / vfree	İlgili serbest bırakma fonksiyonları.

```

1 #include <linux/slab.h>
2
3 /* GFP_KERNEL: normal ayırma; uyuyabilir (bloklayabilir).
4    GFP_ATOMIC: uyumaz; kesme bağlamında / spinlock tutarken kullan. */
5 char *buffer = kmalloc(size, GFP_KERNEL);
6 if (!buffer)
7     return -ENOMEM;                                /* ayırma başarısız */
8
9 /* Sıfırlanmış bellek (calloc benzeri) -- tercih edilen */
10 struct my_data *data = kzalloc(sizeof(*data), GFP_KERNEL);
11 if (!data)
12     return -ENOMEM;
13
14 /* Büyük tampon (fiziksel bitişiklik gerekmiyorsa) */
15 void *big = vmalloc(10 * 1024 * 1024);              /* 10 MB */
16
17 /* Serbest bırak (doğru eşleşen fonksiyonla!) */
18 kfree(buffer);
19 kfree(data);
20 vfree(big);

```


Dikkat

GFP bayrağını doğru seç: GFP_KERNEL bellek beklerken *uyuyabilir*; bu yüzden kesme işleyicide veya spinlock tutarken kullanılamaz — orada GFP_ATOMIC gerekir (uyumaz ama başarısız olma olasılığı daha yüksektir). Yanlış bağlamda GFP_KERNEL kullanmak kilitlenmeye (deadlock) yol açar.

9.2 Kaynak Yönetimli (devm) Ayırma

Modern sürücüler, hata yollarını basitleştirmek için `devm_*` fonksiyonlarını kullanır: bunlar aygıtı “bağlı” ayırmalardır ve aygıt kaldırıldığında *otomatik* serbest bırakılır. Böylece `kfree` çağrılarını unutma riski ortadan kalkar.

```
1 /* Aygıt kaldırıldığında otomatik serbest bırakılır -- kfree gerekmez! */
2 struct my_data *data = devm_kzalloc(dev, sizeof(*data), GFP_KERNEL);
3 if (!data)
4     return -ENOMEM;
5 /* ... temizlik kodunda kfree(data) YOK; çekirdek halleder */
```

10. Eşzamanlılık ve Kilitleme

Çekirdek doğası gereği eşzamanlıdır: aynı sürücü kodu birden çok CPU’da, birden çok süreç tarafından, hatta bir kesmeyle yarıda kesilerek aynı anda çalışabilir. Paylaşılan veriyi korumadan bırakırsan *yarış koşulları* (race conditions) oluşur — veri bozulur, sistem çöker. Kilitleme bunu önler.

10.1 Yarış Koşulu Neden Oluşur?

`counter++` gibi “basit” bir işlem bile aslında üç adımdır (oku, artır, yaz). İki CPU aynı anda yaparsa güncellemelerden biri kaybolabilir. Çözüm: kritik bölgeyi (paylaşılan veriye eriştiğin kod) bir kilit korumak.

Mekanizma

Ne zaman?

mutex	Uyuyabilen bağlam (süreç); kritik bölge uzun/uyuyabilir.
spinlock	Kesme bağlamı veya kısa kritik bölge; uyuyamaz.
atomic_t	Tek bir tam sayı sayaç/bayrak; kilitsiz.
RCU	Çok okuyan, az yazan; yüksek performans.

10.2 Mutex

Mutex, kritik bölgeyi tutan iş parçacığı uyuyabildiğinde kullanılır (örneğin içinde `copy_to_user` veya `kmalloc(GFP_KERNEL)` olan bölge).

```
1 #include <linux/mutex.h>
2
3 static DEFINE_MUTEX(my_lock); /* statik mutex tanımla ve ilklendir */
4
5 static ssize_t my_write(struct file *file, const char __user *buf,
6                        size_t count, loff_t *offset)
7 {
8     if (mutex_lock_interruptible(&my_lock)) /* kilidi al (sinyalle kesilebilir) */
9         return -ERESTARTSYS;
```

```

10
11  /* --- kritik bölge: paylaşılan tampona güvenle eriş --- */
12  if (copy_from_user(device_buffer, buf, count)) {
13      mutex_unlock(&my_lock);          /* hata yolunda da kilidi bırak! */
14      return -EFAULT;
15  }
16  buffer_len = count;
17  /* --- kritik bölge sonu --- */
18
19  mutex_unlock(&my_lock);              /* kilidi bırak */
20  return count;
21 }

```

10.3 Spinlock

Spinlock, kilidi alamayan CPU'nun *uyumadan bekleyip döndüğü* (busy-wait) bir kilittir. Kesme işleyicilerinde veya çok kısa kritik bölgelerde kullanılır — çünkü orada uyumak yasaktır.

```

1  #include <linux/spinlock.h>
2
3  static DEFINE_SPINLOCK(my_spinlock);
4  static int shared_counter;
5
6  void update_counter(void)
7  {
8      unsigned long flags;
9
10     /* irqsave: kilidi al VE bu CPU'daki kesmeleri kapat.
11        Kesmeyle paylaşılan veri için şart (deadlock önler). */
12     spin_lock_irqsave(&my_spinlock, flags);
13
14     shared_counter++;          /* çok kısa kritik bölge */
15
16     spin_unlock_irqrestore(&my_spinlock, flags); /* bırak + kesmeleri geri aç */
17 }

```

Dikkat

Spinlock tutarken asla uyuma! Kilit tutulurken `kmalloc(GFP_KERNEL)`, `copy_to_user`, `mutex_lock` gibi uyuyabilen hiçbir çağrı yapılamaz — yaparsan kilitlenme olur. Ayrıca kritik bölgeyi olabildiğince kısa tut, çünkü bekleyen CPU boşa döner (güç ve performans israfı).

10.4 Atomik İşlemler

Sadece tek bir sayacı korumak için tam bir kilit gereksiz yüküdür. Atomik tipler, bölünmez okuma-değiştirme-yazma işlemleri sunar.

```

1  #include <linux/atomic.h>
2
3  static atomic_t open_count = ATOMIC_INIT(0);
4
5  static int my_open(struct inode *inode, struct file *file)
6  {
7      atomic_inc(&open_count);          /* kilitsiz, güvenli artırma */
8      pr_info("aygıt %d kez açık\n", atomic_read(&open_count));
9      return 0;
10 }

```

```

11
12 static int my_release(struct inode *inode, struct file *file)
13 {
14     atomic_dec(&open_count);
15     return 0;
16 }

```

11. Kesmeler (Interrupts)

Donanım, ilgi istediğinde (veri hazır, tuşa basıldı...) CPU'ya bir *kesme* (interrupt) sinyali gönderir. Sürücü, bu kesmeye bir *işleyici* (handler — ISR) kaydeder. Kesme geldiğinde çekirdek, o an ne yapıyorsa duraklatıp işleyiciyi çalıştırır. Bu, donanımı sürekli yoklamaktan (polling) çok daha verimlidir.

11.1 Kesme İşleyicisi Kaydetme

```

1  #include <linux/interrupt.h>
2
3  static int irq_number;
4
5  /* Kesme işleyicisi (ISR). ATOMİK bağlamda çalışır: uyuyamaz,
6     kmalloc(GFP_KERNEL)/copy_to_user çağıramaz, kısa olmalı. */
7  static irqreturn_t my_irq_handler(int irq, void *dev_id)
8  {
9     /* En kısa işi burada yap (donanımı onayla, bayrak set et) */
10    pr_debug("kesme alındı: irq=%d\n", irq);
11
12    return IRQ_HANDLED;          /* kesme bizim; işlendi */
13 }
14
15 static int __init my_init(void)
16 {
17     irq_number = 42;            /* gerçekte donanımdan/DT'den alınır */
18
19     /* İşleyiciyi kaydet */
20     int ret = request_irq(irq_number, my_irq_handler,
21                          IRQF_SHARED, /* IRQ hattı paylaşılabilir */
22                          "mydev", &my_dev_id);
23
24     if (ret)
25         return ret;
26     return 0;
27 }
28
29 static void __exit my_exit(void)
30 {
31     free_irq(irq_number, &my_dev_id); /* işleyiciyi kaldır */
32 }

```

11.2 Üst Yarı ve Alt Yarı (Top/Bottom Half)

Kesme işleyicisi kısa olmalıdır — çalışırken diğer kesmeler gecikir. Bu yüzden iş ikiye bölünür: **üst yarı** (top half — gerçek ISR) yalnızca acil, kısa işi yapar; ağır iş **alt yarıya** (bottom half) ertelenir ve daha güvenli bir bağlamda çalışır.

Mekanizma	Bağlam	Uyuyabilir mi?
Tasklet	Atomik (softirq)	Hayır (eski, artık workqueue tercih edilir)
Workqueue	Süreç (kernel thread)	Evet
Threaded IRQ	Kendi çekirdek ipliği	Evet

```

1 #include <linux/workqueue.h>
2
3 /* Alt yarı: süreç bağlamında çalışır -> uyuyabilir, kmallocc yapabilir */
4 static void my_work_handler(struct work_struct *work)
5 {
6     /* Ağır işi burada güvenli yap (uzun işlem, uyuyan çağrılar...) */
7     pr_info("alt yarı çalışıyor: veri işleniyor\n");
8 }
9
10 static DECLARE_WORK(my_work, my_work_handler);
11
12 /* Üst yarı (ISR): sadece işi zamanla, hemen dön */
13 static irqreturn_t my_irq_handler(int irq, void *dev_id)
14 {
15     schedule_work(&my_work);          /* alt yarını tetikle */
16     return IRQ_HANDLED;
17 }

```

Bilgi

Threaded IRQ modern ve pratik bir yaklaşımdır: `request_threaded_irq()` ile hem hızlı bir üst yarı hem de kendi ipliğinde uyuyabilen bir alt yarı fonksiyonu verirsin. Çekirdek ikisini otomatik yönetir; ayrı workqueue/tasklet kurmana gerek kalmaz.

12. Çekirdekte Zaman

Sürücüler sık sık zamanla ilgilenir: gecikme koyma, zaman aşımı, periyodik iş. Çekirdeğin zaman birimi *jiffy*'dir — sistem açılışından beri geçen zamanlayıcı tik sayısı.

12.1 jiffies ve Gecikmeler

```

1 #include <linux/jiffies.h>
2 #include <linux/delay.h>
3
4 unsigned long start = jiffies;          /* şu anki tik sayısı */
5
6 /* HZ = saniyedeki tik sayısı (genelde 100, 250 veya 1000) */
7 unsigned long timeout = jiffies + 2 * HZ; /* şimdiden 2 saniye sonrası */
8
9 if (time_after(jiffies, timeout))      /* zaman aşımı kontrolü */
10     pr_warn("zaman aşımı!\n");
11
12 /* Kısa MEŞGUL bekleme (uyumaz) -- yalnızca çok kısa, atomik bağlamda */
13 udelay(50);                          /* 50 mikrosaniye */

```

```

14 mdelay(5);                                /* 5 milisaniye (kaçın!) */
15
16 /* UYUYAN bekleme (tercih edilen, süreç bağlamında) -- CPU'yu bırakır */
17 msleep(100);                             /* 100 ms uyu */
18 usleep_range(50, 100);                   /* 50-100 us esnek uyku */

```

Dikkat

udelay/mdelay CPU'yu *meşgul* tutarak bekler (busy-wait) — işlemciyi boşa harcar. Sadece atomik bağlamda ve çok kısa süreler için kullan. Süreç bağlamında her zaman msleep/usleep_range tercih et; bunlar CPU'yu başka işlere bırakır.

12.2 Çekirdek Zamanlayıcıları

Belirli bir süre sonra bir fonksiyonun çağrılmasını istiyorsak zamanlayıcı (timer) kurarız — örneğin bir donanım yanıtı için zaman aşımı.

```

1  #include <linux/timer.h>
2
3  static struct timer_list my_timer;
4
5  /* Zaman dolduğunda çağrılır (softirq bağlamı -> uyuyamaz) */
6  static void my_timer_callback(struct timer_list *t)
7  {
8      pr_info("zamanlayıcı ateşlendi\n");
9      /* Periyodik istiyorsak yeniden kur: */
10     mod_timer(&my_timer, jiffies + HZ);    /* 1 saniye sonra tekrar */
11 }
12
13 static int __init my_init(void)
14 {
15     timer_setup(&my_timer, my_timer_callback, 0); /* ilklendir */
16     mod_timer(&my_timer, jiffies + HZ);         /* 1 sn sonra ateşle */
17     return 0;
18 }
19
20 static void __exit my_exit(void)
21 {
22     del_timer_sync(&my_timer);                /* zamanlayıcıyı güvenli durdur */
23 }

```

İpucu

Daha yüksek çözünürlük gerekiyorsa hrtimer (high-resolution timer) nanosaniye hassasiyeti sunar. Periyodik ve uyuyabilen işler için ise delayed_work (gecikmeli workqueue) genelde en pratik çözümdür — hem zamanlar hem de süreç bağlamında çalışır.

KISIM 4

Aygıt Modeli ve Sürücü Çerçevesi

13. sysfs ile Kullanıcı Arayüzü

sysfs (/sys), çekirdeğin iç durumunu dosya-benzeri bir hiyerarşi olarak kullanıcı alanına sunar. Bir sürücü, *nitelik* (attribute) dosyaları oluşturarak ayarlarını ve durumunu cat/echo ile erişilebilir kılar — ioctl'e göre çok daha keşfedilebilir ve betiklenebilir.

13.1 sysfs Niteliği Oluşturma

```

1  /* Kullanıcı dosyayı OKUDUĞUNDA çağrılır (cat) */
2  static ssize_t value_show(struct device *dev,
3                          struct device_attribute *attr, char *buf)
4  {
5      return sysfs_emit(buf, "%d\n", my_value);  /* güvenli tampon yazımı */
6  }
7
8  /* Kullanıcı dosyaya YAZDIĞINDA çağrılır (echo) */
9  static ssize_t value_store(struct device *dev, struct device_attribute *attr,
10                          const char *buf, size_t count)
11  {
12      int ret = kstrtoint(buf, 10, &my_value);  /* metni int'e çevir */
13      if (ret < 0)
14          return ret;  /* geçersiz girdi */
15      return count;  /* tüketilen bayt sayısı */
16  }
17
18  /* Nitelik: 0644 izinli, "value" adlı, okuma+yazma dosyası */
19  static DEVICE_ATTR_RW(value);
20
21  /* Kaydet (device_create sonrası): -> /sys/.../mydev/value */
22  device_create_file(my_device, &dev_attr_value);

```

```

1  # Artık kabuktan doğrudan erişilir
2  $ cat /sys/class/myclass/mydev/value
3  42
4  $ echo 100 | sudo tee /sys/class/myclass/mydev/value

```

sysfs kuralı

sysfs dosyaları ideal olarak **dosya başına tek bir değer** içermelidir (bir sayı, bir metin) — karmaşık ikili yapılar değil. “Bir dosya, bir değer” kuralı, sysfs’i basit ve betiklenebilir tutar. Karmaşık veri için debugfs veya karakter aygıtı daha uygundur.

14. Platform Sürücüler

PCI veya USB gibi kendini tanıtan (enumerable) veri yollarında aygıtlar otomatik keşfedilir. Ama bir SoC üzerindeki tümleşik çevre birimleri (GPIO, I2C denetleyici, UART...) kendini tanıtmaz — bunlara *platform aygıtı* denir ve varlıkları sisteme ayrıca bildirilir. Gömülü Linux’ta en çok karşılaşılan sürücü türüdür.

14.1 Platform Sürücüsünün İskeleti

Çekirdek, aygıt ile sürücüyü bir *isim* veya *uyumluluk* dizesiyle eşleştirir; eşleşince sürücünün probe fonksiyonu çağrılır.

```

1 #include <linux/platform_device.h>
2 #include <linux/mod_devicetable.h>
3
4 /* Aygıt bulunup sürücüye bağlandığında çağrılır -- init işini burada yap */
5 static int my_probe(struct platform_device *pdev)
6 {
7     struct device *dev = &pdev->dev;
8     dev_info(dev, "aygıt bulundu, başlatılıyor\n");
9
10    /* Kaynakları al (bellek, IRQ, saat...) -- sonraki bölüm */
11    /* devm_* ile ayır: remove'da otomatik temizlenir */
12    return 0;
13 }
14
15 /* Aygıt kaldırıldığında çağrılır -- temizlik */
16 static void my_remove(struct platform_device *pdev)
17 {
18     dev_info(&pdev->dev, "aygıt kaldırıldı\n");
19 }
20
21 /* Device tree "compatible" eşleştirme tablosu */
22 static const struct of_device_id my_of_match[] = {
23     { .compatible = "mycompany,mydevice" },
24     { /* sonlandırıcı boş girdi */ }
25 };
26 MODULE_DEVICE_TABLE(of, my_of_match);
27
28 static struct platform_driver my_driver = {
29     .probe = my_probe,
30     .remove = my_remove,
31     .driver = {
32         .name = "my_driver",
33         .of_match_table = my_of_match,
34     },
35 };
36
37 /* Tüm init/exit kalıbını tek makroyla üret */
38 module_platform_driver(my_driver);

```

Bilgi

`module_platform_driver()` makrosu, `module_init/ module_exit` fonksiyonlarını ve sürücü kayıt/kayıt-silme çağrılarını otomatik üretir — basit sürücülerde onlarca satır boilerplate'i tek satıra indirir. Benzer makrolar `module_i2c_driver`, `module_pci_driver` için de vardır.

15. Device Tree ve Kaynaklar

Device Tree (DT), donanımın *nerede* olduğunu (adresler, kesme numaraları, saatler) koddan ayrı, bildirimsel bir dosyada tarif eder. Böylece aynı sürücü, farklı kartlarda farklı adreslerle — kodu değiştirmeden — çalışır. Gömülü sistemlerde donanım tanımının standart yoludur.

15.1 Device Tree Dügümü

```
1  /* Device Tree kaynağı (.dts) -- donanımı tarif eder, kod değil */
2  mydevice@10000000 {
3      compatible = "mycompany,mydevice"; /* sürücüyle eşleşen dize */
4      reg = <0x10000000 0x1000>;        /* register adresi + boyut */
5      interrupts = <0 42 4>;              /* kesme numarası ve tipi */
6      clocks = <&clk_controller 5>;       /* kullandığı saat */
7      my-custom-property = <100>;        /* sürücüye özel ayar */
8  };
```

15.2 probe İçinde Kaynak Alma

Sürücü, DT'de tanımlı kaynakları probe sırasında okur. `devm_*` sürümleri, aygıt kaldırılınca kaynakları otomatik serbest bırakır.

```
1  static int my_probe(struct platform_device *pdev)
2  {
3      struct device *dev = &pdev->dev;
4      void __iomem *regs;
5      int irq, ret;
6      u32 custom;
7
8      /* 1) Register bellek bölgesini al ve eşle (MMIO) */
9      regs = devm_platform_ioremap_resource(pdev, 0);
10     if (IS_ERR(regs))
11         return PTR_ERR(regs);
12
13     /* 2) Kesme numarasını al */
14     irq = platform_get_irq(pdev, 0);
15     if (irq < 0)
16         return irq;
17
18     ret = devm_request_irq(dev, irq, my_irq_handler, 0, "mydev", pdev);
19     if (ret)
20         return ret;
21
22     /* 3) DT'den özel bir özelliği oku */
23     if (of_property_read_u32(dev->of_node, "my-custom-property", &custom) == 0)
24         dev_info(dev, "özel değer: %u\n", custom);
25
26     return 0;
27 }
```


15.3 Donanım Register'larına Erişim (MMIO)

Çoğu çevre birimi, register'ları bellek adreslerine eşlenmiş olarak sunar (Memory-Mapped I/O). Bu register'lara *asla* doğrudan işaretçiyle değil, özel erişim fonksiyonlarıyla (`readl/writel`) erişilir — bunlar doğru bariyerleri ve sıralamayı garanti eder.

```

1  #include <linux/io.h>
2
3  #define REG_CONTROL  0x00          /* register offsetleri */
4  #define REG_STATUS   0x04
5  #define REG_DATA     0x08
6
7  /* 32-bit register oku */
8  u32 status = readl(regs + REG_STATUS);
9
10 /* 32-bit register'a yaz */
11 writel(0x1, regs + REG_CONTROL);    /* aygıtı etkinleştir */
12
13 /* Bir durum bitini bekle (zaman aşımıyla) */
14 u32 val;
15 ret = readl_poll_timeout(regs + REG_STATUS, val,
16                          (val & BIT(0)), /* koşul: bit 0 set olana kadar */
17                          10, 1000);    /* 10us aralık, 1000us zaman aşımı */
18 if (ret)
19     dev_err(dev, "aygıt hazır olmadı\n");

```

Dikkat

Donanım register'larına normal işaretçiyle `*(u32*)addr` erişmek yanlıştır: derleyici erişimleri yeniden sıralayabilir, ön belleğe alabilir veya birleştirebilir — donanımla iletişimi bozar. Her zaman `readl/writel` (ve 8/16 bit için `readb/readw`) kullan; bunlar volatile erişim ve bellek bariyerlerini garanti eder.

KISIM 5

İleri Konular ve Hata Ayıklama

16. Bekleme Kuyrukları ve Bloklamalı G/Ç

Çoğu zaman `read`, veri henüz hazır değilken çağrılır. Sürücü ya hemen “veri yok” dönmeli (bloklamasız) ya da veri gelene kadar süreci *uyutmalıdır* (bloklamalı). İkincisi için *bekleme kuyrukları* (wait queues) kullanılır: süreç uyur, CPU’yu harcamaz; veri gelince uyandırılır.

16.1 Bekleme Kuyruğu ile Uyutma ve Uyandırma

```

1  #include <linux/wait.h>
2  #include <linux/sched.h>
3
4  static DECLARE_WAIT_QUEUE_HEAD(read_queue);
5  static bool data_ready;
6
7  /* OKUMA tarafı: veri yoksa uyu */
8  static ssize_t my_read(struct file *file, char __user *buf,
9                        size_t count, loff_t *offset)
10 {
11     /* Bloklamasız istendiyse ve veri yoksa hemen dön */
12     if ((file->f_flags & O_NONBLOCK) && !data_ready)
13         return -EAGAIN;
14
15     /* data_ready true olana kadar UYU (sinyalle kesilebilir) */
16     if (wait_event_interruptible(read_queue, data_ready))
17         return -ERESTARTSYS;          /* sinyal geldi */
18
19     /* Buraya gelindi: veri hazır -> kullanıcıya kopyala */
20     if (copy_to_user(buf, device_buffer, count))
21         return -EFAULT;
22     data_ready = false;
23     return count;
24 }
25
26 /* ÜRETİCİ taraf (ör. kesme işleyici): veri hazır, uyuyanları uyandır */
27 static void data_arrived(void)
28 {
29     data_ready = true;
30     wake_up_interruptible(&read_queue); /* bekleyen okuyucuları uyandır */
31 }

```

Bilgi

`wait_event_interruptible` bir *koşul* bekler ve koşul sağlanana dek uyur; sahte uyanmalara (spurious wakeup) karşı koşulu döngüde tekrar kontrol eder — bu yüzden onu elle döngüye almana gerek yoktur. “interruptible” sürümü, sürecin sinyalle (Ctrl+C) uyandırılabilmesini sağlar; bu neredeyse her zaman tercih edilir (aksi halde süreç öldürülemez).

16.2 poll/select Desteği

Bir uygulamanın birden çok aygıtı aynı anda `poll/select/epoll` ile beklemesi için sürücü `poll` fonksiyonunu sağlamalıdır.

```

1 #include <linux/poll.h>
2
3 static __poll_t my_poll(struct file *file, struct poll_table_struct *wait)
4 {
5     __poll_t mask = 0;
6
7     poll_wait(file, &read_queue, wait);    /* kuyruğu poll tablosuna kaydet */
8
9     if (data_ready)
10         mask |= POLLIN | POLLRDNORM;    /* okunacak veri var */
11
12     return mask;    /* hazır olan olayları bildir */
13 }
14 /* file_operations: .poll = my_poll, */

```

17. Hata Ayıklama Teknikleri

Çekirdekte hata ayıklamak kullanıcı alanından zordur: `gdb` ile adım adım ilerlemek genelde pratik değildir. Bu yüzden çekirdek, kendine özgü zengin hata ayıklama araçları sunar.

Araç	Kullanım
<code>dmesg / printk</code>	En temel; mesajlarla izleme.
<code>dynamic debug</code>	<code>pr_debug</code> 'ı çalışırken aç/kapa.
<code>ftrace</code>	Fonksiyon çağrı izleme, zamanlama.
<code>kgdb / kdb</code>	Çekirdek düzeyinde kesme noktalı hata ayıklama.
KASAN	Bellek hatası dedektörü (use-after-free, taşma).
<code>debugfs</code>	Hata ayıklama için serbest biçimli sysfs.

17.1 Kernel Oops ve Panic Okuma

Bir sürücü hatası (örneğin `NULL` işaretçi dereference), *oops* üretir — çekirdek bir yığın izi (stack trace) ve register durumu basar. Bunu okumak, hatanın nerede olduğunu bulmanın anahtarıdır.

```

1 # Tipik bir oops mesajı (dmesg'de)
2 BUG: kernel NULL pointer dereference at 0000000000000000
3 RIP: 0010:my_read+0x1a/0x80 [mymod]    # <- HATA BURADA: my_read, +0x1a offset
4 Call Trace:

```

```

5  vfs_read+0x9d/0x150          # çağrı yığını (aşağıdan yukarı okunur)
6  ksys_read+0x67/0xe0
7  do_syscall_64+0x59/0x90
8
9  # Ofseti kaynak satırına çevir (hata ayıklama sembolleriyle)
10 $ addr2line -e mymod.ko 0x1a
11 /home/user/mymod.c:42        # tam olarak 42. satır!

```

17.2 debugfs ile İç Durum Sunma

```

1  #include <linux/debugfs.h>
2
3  static struct dentry *debug_dir;
4  static u32 error_count;          /* izlemek istediğimiz sayaç */
5
6  static int __init my_init(void)
7  {
8      /* /sys/kernel/debug/mydev/ altında dosyalar oluştur */
9      debug_dir = debugfs_create_dir("mydev", NULL);
10     debugfs_create_u32("error_count", 0644, debug_dir, &error_count);
11     return 0;
12 }
13
14 static void __exit my_exit(void)
15 {
16     debugfs_remove_recursive(debug_dir); /* tümünü temizle */
17 }

```

İpucu

Geliştirme sırasında debugfs paha biçilmezdir: bir sayaç veya bayrağı, tek satırla (`debugfs_create_u32`) kabuktan izlenebilir hale getirir. sysfs'in aksine debugfs'in "kararlı ABI" garantisi yoktur — yani kuralları gevşektir ve tam da hata ayıklama için tasarlanmıştır. Üretim arayüzleri için sysfs, hata ayıklama için debugfs kullan.

18. Kapanış: Sürücü Geliştirme Yol Haritası

Bu rehberde bir Linux sürücüsünün tüm temel parçalarını gördük. Gerçek bir sürücü, bu parçaların belirli bir donanım için birleştirilmesidir.

İhtiyaç	Kullanılan mekanizma
Modülü yükle/kaldır	module_init / module_exit
Kullanıcı alanına arayüz	Karakter aygıtı (cdev + file_operations)
Komut gönderme	ioctl veya sysfs niteliği
Otomatik /dev düğümü	class_create + device_create
Bellek ayırma	kzalloc / devm_kzalloc
Paylaşılan veri koruma	mutex / spinlock / atomic_t
Donanım olayları	Kesme işleyici + alt yarı (workqueue)
Zaman aşımı / periyodik iş	Zamanlayıcı / delayed_work
Gömülü donanım keşfi	Platform sürücüsü + Device Tree
Bloklamalı okuma	Bekleme kuyruğu + poll
Hata ayıklama	dmesg, ftrace, debugfs, KASAN

İpucu

Sürücü geliştirmeyi öğrenmenin en iyi yolu **gerçek kod okumaktır**: çekirdek kaynağındaki `drivers/` dizini, her tür donanım için binlerce örnek sürücü içerir. Basit bir örnekle (`drivers/char/` altında) başla, onu bir sanal makinede derleyip değiştir. Ayrıca çekirdek belgeleri (`Documentation/` dizini) ve “Linux Device Drivers” (LDD3) kitabı temel başvurulardır. Küçük bir karakter sürücüsü yaz, `dmesg` ile izle, kır ve düzelt — öğrenmenin tek yolu budur. İyi kodlamalar!